Software Requirements Engineering

Requirements Development

Zhiyuan Wan

September 23, 2026



Project

Project Evaluation

- **A group project of developing a software system in teams of 5 members.**
- **There will be 5 milestones requiring group and individual reports** (documents, GitHub repository, commit history, etc).

Your mark will be determined by

1) the quality of your deliverables of milestones
2) version control
3) how well your project is managed (project management)
4) your mid-term and final presentations

Milestones in the Project

ID	Milestone	Percentage	Deliverable (ZIP, PDF)	Due
1	Team Workflow	15%	**1. Document: (PDF)** <= 5 pages, transfer markdown to PDF **2. Process files: (ZIP)** Snapshots of online meetings, meeting minutes, GitHub repo **3. Presentation: (PDF)** <= 10 pages, transfer slides to PDF	24:00 Oct 10, 2026
2	Project Proposal Presentation (Week 3 & 4 - **Oct 9/Sep 29, 2026**) **Vision and Scope & Feature Roadmap**	5%		24:00 Oct 10, 2026
3	Software Requirements Specification	15%		24:00 Oct 24, 2026
4	Mid-term Presentation (Week 6 – **Oct 23/Oct 22, 2026**) **Design, Coding and Testing**	35%		24:00 Oct 24, 2026
5	Final Presentation (Week 8) & Final Report	30%		24:00 Nov 9, 2026

Email to: TA's email address before due date;
Naming: PROJECT-MILESTONE[n]-[team_id]-[team_name]

Milestone 1: Team Work Flow

The deliverable is a document, including:

- **Team name:** Figure out one unique name.
- **Team roles:** Outline team roles and responsibilities.
- **Project management:** Give us information about strategy for project management, e.g., team meeting times (weekly meeting, who will arrange the meeting and record meeting minutes)
- **Plan:** Make timeline of action items for each milestone, making sure that buffers are included in your plan and at least two iterations are scheduled for each milestone.

The main purpose of this document is to give you some rules for team process, management, tracking, and goal setting.

Version Control

Steps

- Setup a private GitHub repository
- Invite all the team members in the GitHub repository
- Restrict document and code update access to authorized individuals
- Setup the conventions to commit changes to documentation and code (e.g., format of commit message, committer, version tag)
- Start tracking the progress since the first draft of the first deliverable

Recommended tools

- Version control - Git
- Documentation - Markdown

Project Management

- ❑ Setup a “process” folder in GitHub repository
- ❑ Make plan for the project, and track weekly action items
- ❑ Reach agreement on the communication strategy, e.g., weekly meeting time and format
- ❑ Prepare template for meeting minutes

Recommended tools

- Notion

Project Milestone 1: Introduction

智能体动态记忆管理

项目背景

在长程、多轮对话中，用户状态、计划和偏好会持续变化。普通向量检索虽然能够找到与问题相关的历史信息，但可能同时召回当前状态与已被后续信息更新的历史状态，从而产生错误回答或决策。

本项目基于 LoCoMo 长期多会话对话数据，开发一个支持结构化记忆构建、查询驱动的动态状态解析、有效记忆检索与可视化的 Web 应用。

需求

1. 长期对话记忆构建与可视化

导入 LoCoMo 多会话对话，提取人物、事件、时间和状态等信息并构建结构化 Memory。每条 Memory 至少保留内容、时间、来源会话/轮次等信息，并支持导出为统一 JSON；Web 端应能查看结构化记忆及其原始证据。

2. 动态记忆更新与有效记忆检索

给定一个问题，系统先从长期历史中检索相关 Memory，再判断这些 Memory 之间的保留、更新/替代或矛盾关系，确定在当前问题与时间条件下仍然有效的 Memory，并据此完成回答；Web 端应支持查看相关历史 Memory 及其关系。

交付形式：可运行的 Web 应用、源代码、结构化 Memory JSON、测试结果和项目报告

数据来源与评测要求

数据来源

1. LoCoMo 课程数据子集

从官方发布的 10 段长对话中选择 6 段作为课程数据：其中 4 段作为开发集，用于系统设计与调试；2 段作为独立测试集，用于最终评测。其余 4 段可作为扩展评测。每段对话平均包含 588.2 轮、27.2 个会话和约 1.66 万 Token，并提供问答、证据轮次和事件摘要标注。

2. 动态记忆测试

课程从所选 LoCoMo 对话的 QA 中统一筛选一组涉及状态变化的问题，并标注与问题相关的历史 Memory、Memory 间的保留 / 更新替代关系，以及最终有效 Memory。学生系统需根据 QA 从完整历史中自动检索相关 Memory，完成关系判断与状态解析后再生成答案。

LoCoMo数据与代码

github.com/snap-research/locomo

评测指标

- 问题回答 F1
- 证据记忆检索 Recall@K
- 动态状态更新判断准确率

对照方法

1. Full Text：全文
2. Vector RAG：检索相关 Memory，但不进行状态关系判断

参考文献

1. Maharana A, et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. ACL 2024.

任务要求与交付

必做要求

1. 数据与记忆构建

使用课程选定的 6 段 LoCoMo 对话（4 段开发 + 2 段测试）；从原始对话中构建结构化 Memory，保存内容、时间、来源等信息，并支持统一 JSON 导出，并实现web端的可视化。

2. 动态维护与查询

针对输入 QA 检索相关历史 Memory，判断 Memory 之间的 RETAIN（保留）、UPDATE / SUPERSEDE（更新 / 替代），确定当前有效 Memory 并完成回答；Web 中展示相关证据与状态解析过程。

扩展功能（Bonus）

- 更复杂的冲突消解与多版本保留
- Graph Memory / 多跳关系建模
- Memory 合并与压缩
- 自适应检索或更复杂的时间推理

建议 Memory 字段

Memory ID · Content · Entity / Event
Timestamp · Source Session / Turn
Query Relation · Validity / State

交付内容

1. 可运行的 Web 应用　2. 完整源代码　3. 结构化 Memory JSON
4. 实验结果与对照分析　5. 项目报告与课堂展示

示例：Memory 构建与状态关系

1. 结构化 Memory 抽取（LoCoMo 真实数据）

原始对话 · conv-26 / D19:1

Caroline: “I passed the adoption agency interviews last Friday!
This is a big move towards my goal of having a family.”

↓ 结构化抽取

统一 JSON 导出示例

```
{
  "memory_id": "conv26_D19_1",
  "entity": "Caroline",
  "attribute": "adoption_status",
  "value": "passed adoption agency interviews",
  "session_time": "2023-10-22 09:55",
  "time_expression": "last Friday",
  "source": {"session": 19, "turn": "D19:1"},
  "state": "active"
}
```

说明：字段设计可由各组扩展；课程只要求结构化、可追溯，并能统一导出 JSON。

真实 LoCoMo：conv-26 D2:8 / D13:1 / D19:1；conv-30 Gina–Jon。

2. Memory 状态关系示例

UPDATE 状态演化 · conv-26 / Caroline

D2:8 researching adoption agencies
↓
D13:1 applied to adoption agencies
↓
D19:1 passed adoption agency interviews

处理：最新阶段为当前有效状态；前序阶段保留为历史 Memory。

SUPERSEDE 状态替代 · conv-30 / Gina

20 Jan 2023 lost her job at DoorDash
↓
16 Mar 2023 online clothes store is open

处理：针对“当前职业/业务状态”查询，旧状态被新状态替代；历史事实仍可追溯。

CONFLICT 状态冲突

Memory A：Gina lost her job at DoorDash.
Memory B：Gina still works at DoorDash.

处理：两条当前事实不可同时成立且无充分证据消解 → 标记 CONFLICT，暂时并存并保留来源。

参考方法：DimMem 与 MRAgent Pipeline

两种长期对话 Memory 方法：结构化混合检索 vs. 图上的主动重构

1. DimMem：结构化 Memory + 三路检索

长对话 → Window 切分 / 可选压缩

↓

结构化 Memory 抽取
Content · Type · Time · Location · Reason · Purpose · Keywords

↓

Query Analysis：问题类型 / 时间 / 地点 / 关键词

↓

三路并行 Retrieval

BM25 词法匹配	Dense Embedding all-MiniLM-L6-v2	Dimension 时间 / 地点 / 类型

→ Union + Dedup → Retrieved Memories → LLM Answer

Embedding 是“并行召回通道”之一；DimMem 的关键是显式 Memory 维度也直接参与检索。

2. MRAgent：Cue–Tag–Content Graph + 主动检索

长对话 → Rewrite / Cue / Tag / Topic 抽取

↓

构建 Cue → Tag → Content Graph
Content：Episodic / Semantic / Topic

↓

Question → 初始 Cue 提取 → Graph Cue Matching

↓

Agentic Retrieval / Active Reconstruction

Embedding 粗筛 text-embedding-3-large	LLM 选择 + Graph Traversal 新 Cue / Evidence 累积

↻ Retrieve → Reason → Retrieve → … → Answer

Embedding 主要做候选控制；核心是根据中间证据动态决定“下一步沿图找什么”。

对课程项目的启发

必做：结构化 Memory → Retrieval → RETAIN / UPDATE / SUPERSEDE / CONFLICT → 有效 Memory → QA；Bonus：Graph / 多跳 / 自适应检索。

参考：DimMem — Dimensional Structuring for Efficient Long-Term Agent Memory；MRAgent — Memory is Reconstructed, Not Retrieved